

MOON WARS

Dokumenty projektowe

update54 → update58

Stan wejściowy: commit `828ed14` (update53)

Testy: 2802 asercji · 79 kroków rysowania · 66 w przeglądarce — zielone

Sekcje w `run_tests.js` do 170 — nowe od 171

Opracowanie: 3 września 2026

Spis treści

1. Roadmapa update54 → update58
2. Projekt: karma dowódcy
3. Projekt: He-3, Moon Gate, regiony
4. Projekt: zwłoki i racje
5. Projekt: religie załogi
6. Projekt: lista gończa i piraci
7. Brief graficzny UI

MOON WARS — ROADMAPA update54 → update58

Ustalone z graczem (jj) 2026-09-03, po update53. Stan wejściowy: commit 828ed14 , testy 2802 + 79 + 66, wszystko zielone, sekcje w run_tests.js idą do 170 — **nowe zaczynać od 171.**

KOLEJNOŚĆ

paczka	zakres	dokument
update54 (i dalsze, wg potrzeb)	bugi i optymalizacja z testów gracza na żywo	—
update55	He-3 jako minerał, okno Moon Gate (zablokowane), regiony mapy	projekt-he3-moongate-regiony.md
update56	menu przy ciele (TREAT/VENT/BAG), cztery racje, wybór kto je, szczury na zapach jedzenia	projekt-zwłoki-racje.md
update57	religie: wiara na rekordzie, odmowa VENT, dni modłów, czwarta opcja w eventach	projekt-religie.md
update58	lista gończa, piraci, ciało jako dowód, cele dodatkowe	projekt-lista-goncza.md

Poprawek po testach będzie prawdopodobnie więcej niż jedna paczka — numeracja przesuwa się wtedy w dół, kolejność zostaje.

Karma dowódcy (projekt-karma-dowodcy.md) przewija się przez 56–58, więc jej projekt musi być zatwierdzony **przed** update56. Inaczej trzy paczki dopiszą do niej po kawałku i znowu wyjdzie tabelka.

Brief graficzny (brief-graficzny-ui.md) idzie równolegle — szkice z GPT nie blokują żadnej paczki, a przemalowanie UI zrobimy jako osobną paczkę, gdy kierunek będzie wybrany.

DECYZJE PODJĘTE (żeby nie wracać)

- **He2 zostaje paliwem. He-3 to osobny minerał.** Bez zastosowania w demie. Zero kodu blockchainowego w demie — to sprawa pełnej wersji i poza grą.
- **Moon Gate: miejsce, nie mechanika.** Zakładka GATE , lista części z magazynu bazy, wszystko zablokowane, **bez przycisku BUILD.**
- **Regiony teraz, treść później.** Jeden region luna , trzy kontrakty przypięte, migracja starych zapisów.
- **Worek na zwłoki: 2 pola, kot 1.** Zeruje rozkład.
- **Pochówek płaci KARMA, nie XP.** W XP_RATES nie ma pasującej umiejętności, a dziewiąta rozwaliliby 25 rang z update52.

- **Dzisiejsza racja zostaje na 50 najedzenia.** Obniżenie do 25 podwoiłoby zużycie żarcia w całej grze — to zmiana balansu, nie efekt uboczny religii.
- **Religie: mechanika z praktyki, nigdy ze stereotypu.** Żadna wiara nie ma bonusu do pieniędzy ani handlu. Handel ludźmi zostaje jako mechanika piratów i syndykatów, nieprzypisany do żadnej realnej wspólnoty.
- **Dzień modłów musi mieć zapłatę** (powrót najedzony i wyleczony + karma) — inaczej gracz nauczy się wierzących nie brać i projekt odwróci się przeciw własnemu celowi.
- **Licznik modłów na człowieku, nie na grze** — inaczej cała załoga wychodzi naraz.
- **Lista gończa: jeden rejestr.** Mapa wskazuje na rekord, nie kopiuje go. Pirat znika dopiero po dostarczeniu ciała do bazy.
- **Morale: odłożone.** Nie istnieje w kodzie; to osobny duży update, nie dodatek do religii.
- **Karma w portach: wariant B — NARZUT za niską karmę, bez zniżki za wysoką.** Cena normalna zostaje ceną dzisiejszą. Do tego **odmowa obsługi** przy bardzo niskiej karmie: część portów nie handluje albo pokazuje węższy asortyment. Powód: przy zniżce za dobro zła karma nie kosztowałaby nic.

PYTANIA OTWARTE

1. **Karma / plansza:** ściana ustępuje z jedyne go odkrytego pola, czy pierwszy kwadrat jest zawsze wolny?
2. **~~Karma / porty:** narzut czy zniżka?~~ **ZAMKNIĘTE: narzut + odmowa obsługi.** Zostaje dobranie progów: od jakiej karmy narzut, od jakiej odmowa, i czy odmawiają wszystkie porty czy tylko część.
3. **Karma / walka:** poddawanie się wroga — w tej samej paczce czy osobno?
4. **Lista gończa:** dostarczanie ściganego **żywcem** — w update58 czy później?
5. **Grafika:** cztery kolory stanu — zgoda? Korporacje na glify zamiast kolorów? Tylko skóra, bez przesuwania elementów?
6. **Moon Gate:** ostateczna lista części.

PUŁAPKI DO PILNOWANIA W KAŻDEJ Z TYCH PACZEK

- Dwie kopie tej samej liczby zawsze się rozjadą — **kasuj jeden rejestr.**
- Kod nieosiągalny z pewnym siebie komentarzem jest gorszy niż brak kodu. Jak gałąź nie da się wywołać z testu, siedzi w złym miejscu.
- Geometria UI z **jednego źródła** (`Renderer.orderRects()` jako wzór).
- Test może przechodzić z niewłaściwego powodu — sprawdzać, czy kod **BY ZADZIAŁAŁ**, gdyby nie poprawka.
- **Ograniczać pętle `while` w testach** — zawieszenie to jedyna awaria, której przebieg łamania nie umie zaraportować.
- Migracje zapisów mają własne testy (wzór: `captain.js` → `commander.js`).

MOON WARS — PROJEKT: KARMA DOWÓDCY

Dokument projektowy, **zero kodu**. Do przegadania przed update56. Stan wejściowy sprawdzony w kodzie 2026-09-03 (commit 828ed14 , update53).

1. CO JEST DZISIAJ (zweryfikowane w źródle, nie z pamięci)

Rejestr. Jedno pole `karma` na rekordzie dowódcy, `0–100`, start 50 (`commander.js:210`). Przeżywa zapis. Nie ma drugiej kopii — to dobrze.

Wszystkie źródła. Tabela `KARMA` w `commander.js:505`, pięć pozycji i nic poza nią:

stała	wartość	kiedy
<code>HELP_AT_COST</code>	+5	pomoc kosztem własnych zasobów
<code>RESCUE_AT_COST</code>	+10	ratunek kosztem wyprawy
<code>ROBBERY</code>	−5	zabranie komuś, kto nie może się bronić
<code>KILL_HELPLESS</code>	−10	dobicie tego, co się poddało
<code>EVACUATE</code>	−10	zostawienie żywej załogi

Wszystkie pięć odpalają **wyłącznie wybory w eventach mapy**. Komentarz w kodzie mówi wprost, dlaczego: „*naprawy, gaszenie, opatrywanie i strzelanie do uzbrojonego wroga to po prostu robota. Karma jest od decyzji O LUDZIACH, KTÓRZY NIE MOGĄ ODDAĆ.*” Ta zasada jest dobra i **zostaje**.

Jedyny skutek. `Chips.wallColumn(karma)` (`chips.js:158`) — pięć progów:

```
karma  0–14  → kolumna 1      (blokada stoi w pierwszej kolumnie)
        15–34 → kolumna 2
        35–65 → kolumna 3      ← start gry
        66–85 → kolumna 4
        86–100 → kolumna 5
```

Kolumna z blokadą jest martwa, na lewo od niej jest strona `good` (chipy Ethos), na prawo `evil` (Dominance), `uni` wchodzi wszędzie (`chips.js:219`). **I to jest wszystko**. Poza `wallColumn` nikt w całej grze nie czyta `karma`.

Efekt uboczny. `shift()` zwraca `killed` — ile chipów przestało działać po przesunięciu ściany. Gracz widzi to PRZED zatwierdzeniem wyboru. Chip po złej stronie gaśnie i **wraca sam**, gdy ściana się cofnie.

2. DIAGNOZA — czego brakuje

Słowa gracza sprzed update51: „*męczy mnie ta mechanika kapitana z karmą, coś tam brakuje*”. Po przeczytaniu kodu widzę cztery konkretne braki. To nie jest bug — to jest niedokończony projekt.

2.1. Karma jest suwakiem, nie pamięcią

Ma dokładnie **jednego odbiorcę** — pozycję ściany na planszy chipów. Wszystko, co gracz robi przez cały przelot, przechodzi przez tę jedną rurę i wychodzi jako „masz o jeden slot więcej po lewej”. To za wąsko na mechanikę, która nazywa się karmą i której gracz spodziewa się w każdym oknie.

2.2. Świat nie reaguje

Żaden port, żaden rekrut, żaden wróg, żaden kontrakt nie wie, kim jesteś. Bezwzględny dowódca i święty mają te same ceny, tych samych chętnych do służby i tych samych przeciwników. Karma bez świadków to nie karma, to statystyka.

2.3. Kara jest odwracalna i niewidoczna

Chip po złej stronie gaśnie i wraca sam. Czyli zła decyzja to **chwilowe wyłączenie bonusu**, nie konsekwencja. Nic w grze nie pamięta, że to się stało.

2.4. Uczciwy skraj: poziom 1 + karma ≤ 14 = zero pól

Dowódca poziomu 1 ma jeden odkryty kwadrat (jeden na poziom, update52), a ściana przy karmie ≤ 14 stoi w kolumnie 1 — czyli dokładnie w nim. Test to dziś stwierdza wprost. Gracz, który zacznie od dwóch złych decyzji, dostaje planszę, na której nie da się nic położyć, i **nie dowiaduje się dlaczego**.

3. PROPOZYCJA

Trzy zmiany. Każda używa rejestrów, które już są.

3.1. Karma płaci za to, co robisz na statku — nie tylko za klikanie w okienku

Eventy mapy zostają, dochodzą źródła z rozgrywki. Zasada z komentarza obowiązuje bez wyjątku: **liczą się decyzje o tych, którzy nie mogą oddać**.

zdarzenie	delta	dla czego to karma, a nie robota
pochówek swojego zmarłego w bazie (BAG → MEMORIAL)	+3	trup nie może się o to upomnieć
wyrzucenie zwłok przez służbę mimo sprzeciwu wiary	-3	j.w., w drugą stronę
puszczenie załoganta na modły (dzień odpoczynku)	+2	kosztuje Cię rękę na kontrakcie
zabranie go mimo święta	-2	j.w.
dobicie poddanej załogi abordażowej	-10	istniejące KILL_HELPLESS , tylko wpięte też w walkę
wzięcie jeńca zamiast dobicia	+5	kosztuje miejsce i ryzyko
dostarczenie ściganego żywego zamiast ciała	+5	patrz projekt listy gończej

Ważne: **jedna decyzja punktuje raz**. Dziś pilnuje tego miejsce, w którym wybór jest konsumowany, nie `shift()` — i tak ma zostać.

3.2. Świat czyta karmę — cztery odbiorcy, wszyscy istniejący

To jest sedno tej przebudowy. Karma przestaje być suwakiem planszy i staje się tym, kim jesteś dla innych.

1. **Rekruci w bazie** (`Base.hireRecruit`) — wysoka karma: więcej chętnych, taniej. Niska: mało kto chce, drożej, bo płacisz za milczenie.
2. **Porty i stacje** (`station.js` , ceny w `base.js`) — **WARIANT B, decyzja gracza**: cena normalna jest ceną dzisiejszą i płaci ją każdy od średniej karmy w górę. **Niska karma dokładnie narzut**. Żadnej zniżki za wysoką karmę.

Powód: przy niższej za dobro drań płaci tyle co dziś, czyli **zła karma nie kosztuje nic** — dostaje chipy Dominance, haracz i prawą stronę planszy za darmo. Dobra karma płaci swoje gdzie indziej (odpuszczony łup, brak haraczu, brak Dominance); zła nie płaciła nigdzie. Narzut to naprawia.

Do tego: nie każdy port musi Cię obsłużyć. Przy bardzo niskiej karmie stacja może odmówić handlu albo pokazać węższy asortyment. To boli mocniej niż +2 CC i nie wymaga majstrowania przy cenach. Progi odmowy i asortymentu do wyważenia po pierwszym teście — **odmowa musi mieć widoczny powód na ekranie**, inaczej gracz przeczyta ją jako bug. 3. **Wróg w walce** — wysoka karma: przeciwnik częściej się poddaje (i wtedy decyduje `KILL_HELPLESS` naprawdę stoi przed graczem). Niska: nie poddaje się nigdy, bo wie, co go czeka — walczy do końca, dłuższe i droższe walki. 4. **Kontrakty** — część zleceń widoczna tylko powyżej/poniżej progu.

Obie skrajności mają być grywalne. Nie robimy „dobrego zakończenia”: - **wysoka** — ludzie, ceny, poddania się, chipy Ethos; - **niska** — haracz (`Extortion` już jest w `CHIP_DEFS`), strach, twarde walki, chipy Dominance; - **środek 35-65** — najgorsze z obu i najszersza plansza. To ma być wybór, a nie bezpieczna przystań.

3.3. Naprawa uczciwego skraju

Ściana **nigdy nie stoi w jedynym odkrytym kwadracie**. Jeśli dowódca ma tylko kolumnę 1 odkrytą, a karma mówi „ściana w 1”, ściana przesuwa się o jedną w prawo i plansza pokazuje o tym napis. Alternatywa: pierwszy kwadrat jest zawsze **uni** i wolny od ściany. Do wyboru przy implementacji — **decyzja gracza (jj)**.

4. CZEGO NIE ROBIMY

- **Nie dodajemy drugiego rejestru.** Żadnej „reputacji” obok karmy. Jedna liczba, wielu czytelników.
 - **Nie robimy dryfu w stronę 50.** Karma jest pamięcią, nie temperaturą.
 - **Nie karzemy za robotę.** Naprawa, gaszenie, strzelanie do uzbrojonego wroga to nadal zero.
 - **Nie odbieramy chipów na stałe.** Gaśnięcie i powrót zostają — konsekwencją ma być świat z §3.2, nie utrata przedmiotu.
-

5. PYTANIA DO GRACZA

1. Naprawa skraju: ściana ustępuje z jedynego pola, czy pierwszy kwadrat jest zawsze wolny?
2. ~~Narzut czy zniżka w portach?~~ **ZAMKNIĘTE: wariant B — narzut za niską karmę, plus odmowa obsługi przy bardzo niskiej.**
3. Poddania się wroga — nowa mechanika w walce. Robimy w tej paczce, czy osobno?
4. Progi: od jakiej karmy narzut, od jakiej odmowa obsługi? Czy odmawiają wszystkie porty, czy tylko część (żeby nie zablokować przelotu na amen)?

MOON WARS — PROJEKT: He-3, MOON GATE, REGIONY (update55)

Dokument projektowy, **zero kodu**. Stan wejściowy sprawdzony w kodzie 2026-09-03 (commit 828ed14 , update53).

Cel tej paczki: PRZYGOTOWAĆ MIEJSCE, nie dodać rozgrywki. Wszystko poniżej ma w demie być widoczne i zablokowane. Treść wchodzi w pełnej wersji.

1. He-3 (Helium-3)

1.1. Uwaga na kolizję — w grze JEST już „He2”

Paliwo nazywa się dziś **He2** i siedzi w ośmiu plikach: kanistry w ładowni (`ship.js:1737`), pasek na HUD (`renderer.js:775` , `:1428`), magazyn i cena 8 CC (`base.js:106`), sklep stacji (`ui.js:606`), spalanie przy ucieczce (`combat.js:222`), eventy mapy (`map.js:136` , `:145`).

Decyzja gracza: He2 zostaje paliwem bez zmian. He-3 to OSOBNY minerał. Nie przemianowujemy, nie łączymy, nie robimy z paliwa waluty. Dwa różne surowce, dwa różne zastosowania, zero wspólnych liczb.

1.2. Czym He-3 jest w demie

Minerałem obok pozostałych i niczym więcej. Wydobywa się / znajduje / kupuje, leży w ładowni i w magazynie, ma cenę odsprzedaży. **Nie ma zastosowania.** Opis itemu mówi wprost, że to surowiec strategiczny i że przyda się później — to jest zapowiedź, nie ślepy zaulek.

1.3. Czym He-3 będzie w pełnej wersji (kontekst, NIE robimy teraz)

- **Główny token gry** — paliwo Moon Gate'a: skok do innego księżyca kosztuje He-3.
- Dalsze zastosowania (skiny itp.) — do zaprojektowania osobno.
- Ewentualne powiązanie z tokenem na WAX to **sprawa poza kodem gry** i poza demem. W demie nie ma i nie będzie żadnego kodu blockchainowego, żadnego portfela, żadnego połączenia sieciowego. Demo jest grą offline.

1.4. Zakres pracy w update55

1. Definicja itemu w `cargo.js` obok `ration_pack` — rozmiar, kolor, `stackMax` , `unitValue` , opis.
2. Wejście do magazynu bazy (`base.js`) i do sklepu (`ui.js`) na tych samych zasadach co reszta towaru — **żadnego osobnego rejestru**, He-3 to zwykły item w istniejącej siatce.
3. Źródła: drop z wraków i bossów, towar na stacji. Ilości symboliczne.
4. Wyświetlanie: liczy się jak każdy inny ładunek, **nie dostaje własnego paska na HUD** (pasek jest dla He2, bo He2 decyduje, czy wrócisz).

2. MOON GATE

2.1. Czym jest

Konstrukcja w bazie, budowana z **części pochodzących z istniejących itemów**. Gotowa Brama otwiera loty do innych księżyców Układu. W demie: **widoczna, opisana, zablokowana**.

2.2. Gdzie mieszka

Nowa zakładka w bazie. Dzisiejsze zakładki (`basescreen.js:24`):

HANGAR · ARMOURY · CREW · MESS · SUPPLY · UPGRADES · MEMORIAL

Dochodzi **GATE** na końcu. Zakładki stoją na `40 + i·116` przy szerokości 108 (`basescreen.js:501`), więc ósma zaczyna się na `852` i kończy na `960` — mieści się w kadrze 1280 z zapasem. Bez przebudowy paska.

2.3. Co widać na ekranie

- Rysunek Bramy w budowie (na razie schemat, nie grafika).
- **Lista części z licznikami** `0/N`, czytana z **tego samego magazynu bazy** co reszta gry. Jeden magazyn od `update35` — nie dorabiamy drugiego.
- Każda część wyszarzona, z opisem, skąd pochodzi.
- Jeden napis u dołu: konstrukcja niedostępna w tej wersji.
- **Zero przycisku „BUILD”**. Przycisk, który nic nie robi, jest gorszy niż jego brak — to nasza reguła o nieosiągalnym kodzie, tylko w UI.

2.4. Części — propozycja wstępna (do zatwierdzenia)

Wszystkie z rzeczy, które już w grze istnieją albo naturalnie do niej pasują: rdzeń reaktorowy, cewki nadprzewodzące, płyty kadłubowe, rdzeń nawigacyjny, He-3 w większej ilości. Dokładna lista i liczby — przy pełnej wersji.

3. REGIONY MAPY

3.1. Dlaczego TERAZ, skoro księżycy dopiero później

Dziś mapa nie ma nad sobą żadnej warstwy „gdzie jesteśmy” — trzy kontrakty istnieją bez adresu. Dodanie tej warstwy później oznacza **migrację wszystkich zapisów**, dokładnie taką jak `captain.js` → `commander.js` w `update52`. Zrobione teraz, gdy region jest tylko jeden, kosztuje jedno pole w zapisie.

3.2. Co robimy

1. Pojęcie **regionu** nad mapą sektora. Jeden region: `luna`, etykieta `LUNA`.
2. Trzy dzisiejsze kontrakty przypięte do `luna`. Nic w rozgrywce się nie zmienia.

3. Region **widoczny na ekranie mapy** — gracz od razu wie, że jest gdzieś konkretnie i że są inne miejsca.
4. Miejsce na kolejne regiony w strukturze danych + **jeden rejestr odkrycia** (`unlockedRegions`), z którego korzystać będzie Brama.
5. Migracja starych zapisów: brak regionu → `luna` .

3.3. Czego NIE robimy

Nowych map, nowych wrogów, nowych eventów, przelotów między księżycami. To jest pełna wersja.

4. RYZYKA

- **Podwójny rejestr paliwa.** Jeśli kiedykolwiek He-3 zacznie być zużywany do lotu, natychmiast pojawi się pokusa „to niech He2 też...”. Nie. He2 = lot w sektorze, He-3 = Brama. Rozdzielne na zawsze.
 - **Zakładka GATE jako śmietnik.** Nie wolno do niej wrzucać niczego poza Bramą.
 - **Regiony a zapis.** Migracja musi mieć test — jak przy update52.
-

5. TESTY (nowe sekcje od 171 w górę)

- He-3 jest itemem: wchodzi do ładowni i magazynu, ma cenę, **nie zmienia paska paliwa** ani niczego, co czyta He2.
- He-3 nie ma żadnego zastosowania — próba użycia go jako paliwa jest odmawiana.
- Zakładka GATE rysuje się, prostokąty zakładek się nie nakładają (test rysowania w stylu update52a), lista części czyta magazyn bazy.
- Brak przycisku budowy: żadne kliknięcie w zakładce GATE nie zmienia stanu gry.
- Region: nowy przelot dostaje `luna` ; **stary zapis bez regionu dostaje `luna` po wczytaniu**; trzy kontrakty należą do `luna` .

MOON WARS — PROJEKT: ZWŁOKI I RACJE

(update56)

Dokument projektowy, **zero kodu**. Stan wejściowy sprawdzony w kodzie 2026-09-03 (commit 828ed14 , update53).

1. ZWŁOKI I RANNI — DECYZJA WRACA DO GRACZA

1.1. Co już jest (nie budujemy tego od zera)

element	gdzie	zachowanie
rozkład ciała	ship.js:2115	DECAY_SECONDS = 40 , potem decaying = true
alert	ship.js:1236	„...body is DECAYING — open an airlock and get it OUT!”
noszenie rannego	ship.js:1440	do medyka, kładzie i staje
noszenie zmarłego	ship.js:1454	do najbliższej OTWARTEJ śluzy , wyrzuca
brak śluzy	ship.js:1470	czeka CORPSE_HOLD_SECONDS , potem kładzie i wraca do roboty
wysyłanie po ciało	ship.js:1333	_rescueId , ktoś idzie po gnijącego (update42)
cmentarz	baza, MEMORIAL	istnieje od update35, z historią służby

Czyli cała logistyka stoi. **Zmieniamy jedno: kto decyduje.** Dziś decyduje automat i otwarte drzwi. To jest dokładnie ten sam problem, który rozwiązał update48 przy łupach — „*nic nie znika bez decyzji gracza*”.

1.2. Menu przy leżącym

Zaznaczasz żywego załoganta → klikasz w leżącego (rannego albo zwłoki) → mały panel przy kursorze, trzy przyciski:

TREAT
VENT
BAG

ranny → do medyka (istniejąca ścieżka)
ciało → do śluzy (istniejąca ścieżka)
ciało → do ładowni (NOWE)

- **TREAT** — tylko dla rannego, tylko gdy statek ma medyka.
- **VENT** — zawsze dostępne dla zwłok. Darmowe. Zostaje.
- **BAG** — nosi ciało do ładowni, **zajmuje 2 pola siatki** (kot: 1), **zeruje _rotT**. Trup przestaje się psuć, ale zabiera miejsce na łup.

Bez zaznaczenia nie dzieje się nic. Automatyczne wynoszenie do śluzy znika — ciało leży, aż gracz zdecyduje. Alert o rozkładzie zostaje i staje się tym, czym miał być: zegarem na decyzję.

1.3. Worek jako przedmiot

Nowa definicja w `cargo.js` obok `ration_pack`: `w:2, h:1` (koci `w:1, h:1`), `stackMax: 1`, brak wartości handlowej. Nazwisko zmarłego siedzi na przedmiocie — w bazie ciało schodzi na cmentarz **z tą samą historią służby**, którą cmentarz już umie pokazywać.

1.4. Zapłata

Pochówek w bazie to **+3 karmy** dowódcy (patrz `projekt-karma-dowodcy.md` §3.1). **Nie XP**. Sprawdziłem `XP_RATES` (`crew.js`) — jest tam osiem pozycji i żadna nie pasuje; pochówek to nie `combat` ani `repair`. Dziewiąta umiejętność rozwaliłaby arytmetykę 25 rang z `update52`, bo rangi liczą się z kwadracików. Karma jest właściwym rejestrem i akurat tego jej brakuje.

1.5. Geometria — jedno źródło

Prostokąty menu muszą mieć **jedną funkcję** je opisującą, jak `Renderer.orderRects()` z `update53`. Rysowanie i test trafienia czytają stamtąd. Prostokąt wypisany osobno w dwóch miejscach to ten sam błąd, który sprzątaliśmy przy `BOARD`.

2. RACJE — CZTERY TYPY

2.1. Co jest dzisiaj

`HUNGER` (`crew.js:264`): pasek 0–100, `HUNGRY 40` (głodny sam idzie po żarcie), `STARVING 12` (traci HP `1/6` na sekundę), `EAT_SECONDS 3`. `HUNGER.FOOD`: `ration 50`, `rat 45`, `spider_egg 60`. `ration_pack` w `cargo.js:97`: `w:1, h:1`, `stackMax: 5`, `unitValue: 8`.

2.2. Nowa tabela

racja	najedzenie	slot	cena	mięso?
Protein Paste	25	1	5 CC	tak
Ration Pack (dzisiejsza, bez zmian)	50	1	8 CC	tak
Field Meal	80	2	16 CC	tak
Green Ration	50	1	12 CC	nie — je każdy

Dlaczego dzisiejsza racja zostaje na 50. Propozycja gracza (zwykła 25) oznaczałaby po cichu **podwojenie zużycia żarcia w całej grze** — 40 minut lotu na pełnym pasku robi się 20. To zmiana balansu przemyciona przy okazji religii; lepiej ją zrobić świadomie albo wcale.

Gdzie siedzi różnica. Nie w cenie za punkt najedzenia (ta jest podobna), tylko w **miejscu w ładowni** — a ładownia jest w tej grze prawdziwą walutą od `update24`. `Field Meal` karmi najlepiej i zabiera dwa pola. `Green Ration` jest droższa, ale **zje ją każdy**, więc na mieszanej załodze jest po prostu wygodna. To decyzja przy pakowaniu, nie podatek od wiary.

2.3. Darmowe zazębie

`rat` i `spider_egg` są mięsem. Wegetarianin ich nie tknie — czyli `update45` (koty, szczury) i `update47` (jaja, głód) same z siebie zaczynają grać z religią, bez jednej linijki nowego kodu.

2.4. Wybór, kto je

Dziś głodny sam sięga po rację (`ship.js:2255`), więc racje rozchodzą się bez udziału gracza. **Gracz ma moc wskazać, kto dostaje ostatnią.** Mechanika samodzielna, warta zrobienia niezależnie od religii — i warunek konieczny dla sikhijskiego oddawania racji.

3. DODATEK: JEDZENIE W ŁADOWNI ŚCIĄGA SZCZURY

Opis racji w `cargo.js:102` **już to obiecuje**: „*Keep it stowed: a hold that smells of food does not stay empty.*” Kod tego nie robi.

Żarcie na półce podnosi szansę na szczury w ładowni. Szczury, koty i głód już istnieją — to kilkanaście linijek za mechanikę, która dotyka każdego przelotu: ile jedzenia wiesz, a nie tylko ile miejsca zostało. Kot przestaje być ozdobą, a Field Meal (2 pola, dużo zapachu) dostaje drugą wadę.

4. TESTY (nowe sekcje)

- Ciało **nie jest** wynoszone do śluzy bez rozkazu gracza — nawet przy otwartej śluzie i po `DECAY_SECONDS`.
- TREAT/VENT/BAG: każdy prowadzi do właściwego miejsca; TREAT niedostępny bez medyka i dla zmarłego.
- BAG zajmuje 2 pola, koci 1; **przy pełnej ładowni jest odmawiany, nie zjadany** (reguła z `update53`: rozkaz bez skutku jest odmawiany).
- BAG zeruje `_rotT` — zapakowane ciało nie przechodzi w `decaying`.
- Pochówek w bazie daje +3 karmy **raz** — powtórne wejście na cmentarz nie płaci drugi raz.
- Menu: ten sam prostokąt w rysowaniu i w kliknięciu; żadne dwa przyciski się nie nakładają (test rysowania w stylu `update52a`).
- Cztery racje: wartości najedzenia, rozmiary, ceny; **wegetarianin odmawia mięsnej, szczura i jaja**, ale zje Green Ration.
- Szczury: hold z jedzeniem ma wyższą szansę niż pusty; **pusty hold nie generuje szczurów z powodu tej reguły**.

MOON WARS — PROJEKT: RELIGIE ZAŁOGI

(update57)

Dokument projektowy, **zero kodu**. Stan wejściowy sprawdzony w kodzie 2026-09-03 (commit 828ed14 , update53). Zależy od update56 (racje, menu przy ciele, wybór kto je).

1. PO CO TO JEST

Skolonizowany Księżyc nie jest światem bez wiar — poleciały z ludźmi. Załoga ma je mieć, a gra ma pokazywać, że **ludzie różnych wyznań pracują na jednym statku** i że to działa. To jest cel projektowy i on wyznacza wszystkie decyzje poniżej.

Z tego wynika jedna twarda zasada rzemieślnicza:

Mechanika wiary wychodzi z PRAKTYKI — z tego, co dana religia każe robić. Nigdy ze stereotypu o ludziach, którzy ją wyznają.

Tak robi to CK3: post, dni odpoczynku, zakazy pokarmowe, obrzędy nad zmarłym, pielgrzymka. Nie ma tam linijki „grupa X: +10% do handlu”, bo to nie wynika z żadnej doktryny — to wynika z karykatury. Karykatura w tabelce jest przy okazji **słabszą mechaniką**: modyfikator procentowy zamiast decyzji.

Dlatego w Moon Wars **żadna wiara nie dostaje bonusu do pieniędzy ani do handlu**. Wszystkie płacą w rzeczach, które już mamy: głód i racje, zwłoki i cmentarz, karma, obsada konsol, wybory w eventach.

Handel załogantami zostaje w grze jako mechanika (patrz lista gończa i piraci) — ale nie jest przypisany do żadnej realnej wspólnoty. Handlują nim piraci i syndykaty, czyli frakcje, które istnieją po to, żeby robić złe rzeczy.

2. REJESTR

Jedno pole `faith` na rekordzie załoganta, losowane przy rekrutacji. Przeżywa zapis. Widoczne w **teczce** (mamy ją od update52a) i w panelu załogi. Dowódca ma to samo pole — jego wiara działa na skalę statku.

Migracja: stary zapis bez `faith` → losowanie przy wczytaniu **raz**, nie co klatkę.

3. SZEŚĆ WIAR — CO ROBIA

Wszystko poniżej opiera się o istniejące systemy. W nawiasie plik.

wiara	racje	zwłoki	dzień modłów	event mapy
Judaizm	bez mięsnych	odmawia VENT	tak	dodatkowa opcja
Islam	bez mięsnych	odmawia VENT	tak	dodatkowa opcja
Chrześcijaństwo	wszystko	odmawia VENT	tak	dodatkowa opcja
Hinduizm	bez mięsnych	odmawia VENT	tak	dodatkowa opcja
Buddyzm	bez mięsnych	odmawia VENT	tak	dodatkowa opcja
Bez wyznania	wszystko	zgadza się na wszystko	nie	brak

„Bez wyznania” jest pełnoprawnym wyborem losowania i **nie jest gorszy** — po prostu nie ma ani ograniczeń, ani dodatkowych opcji.

3.1. Racje (zależy od update56)

Wiara określa tylko jedno: czy załogant tknie mięso. Mięsne są `Protein Paste`, `Ration Pack`, `Field Meal`, a także `rat` i `spider_egg` (`crew.js:264`). `Green Ration` je każdy.

To nie jest kara — to jest **powód, żeby pakować zieloną**, i decyzja przy ładowni.

3.2. Zwłoki (zależy od update56)

Wierzący **odmawia VENT** nad ciałem współwyznawcy. Jedna implementacja reguły, tego samego kształtu co `Commander.orderRefusal(key)` z `update53`: pyta się o powód, powód idzie na ekran. Gracz może kazać mimo wszystko — wtedy **−3 karmy**. Pochówek w bazie: **+3 karmy**.

3.3. Dzień modłów

Co trzeci kontrakt załogant zostaje w bazie. Licznik siedzi **na człowieku, nie na grze** — inaczej jednowyznaniowa załoga wychodzi na modły cała naraz i co trzeci kontrakt lecisł pustym statkiem. Własny licznik rozkłada ich sam.

To musi coś dawać, inaczej zepsuje cały projekt. Załogant, który co trzeci kontrakt siedzi w domu, jest czystą stratą — a skoro wiara jest losowana, gracz w trzy przeloty nauczy się wierzących nie brać, nie awansować i sprzedawać przy pierwszej okazji. Wiara stałaby się wadą do unikania, czyli dokładnie odwrotnie do celu z §1.

Zapłata: - wraca **z pełnym paskiem głodu i wyleczony** (za darmo — normalnie płacisz); - dowódca dostaje **+2 karmy** za to, że go puścił; - zabranie go mimo święta: **−2 karmy** i on leci, ale nic nie zyskujesz.

Czyli to jest wybór: rękę mniej na kontrakcie, czy karma w dół.

3.4. Czwarta opcja w evencie mapy

Najtańszy i najmocniejszy kawałek całego projektu. Eventy w `map.js` już mają listę opcji z wynikami — wierzący dowódca (albo obecność współwyznawcy w załodze) **odblokowuje dodatkową opcję**, której inni nie widzą: pomodlić się z rozbitkami, odmówić rabunku miejsca kultu, wynegocjować po swojemu.

Zero nowego kodu w silniku — sama treść. I to jest ten brakujący kawałek karmy z `projekt-karma-dowodcy.md`: karma przestaje być listą punktów, a zaczyna zależeć od tego, **kim jest Twój dowódca**.

4. CZEGO NIE ROBIMY

- **Żadnych bonusów do cen, handlu i pieniędzy przypisanych wierze.**
- **Żadnej wiary jako etykiety frakcji wrogów.** Piraci to piraci.
- **Żadnych konfliktów między wyznaniem w załodze.** Cel jest odwrotny: mają razem latać. Ewentualne tarcia wracają dopiero z systemem morale, jeśli kiedykolwiek powstanie.
- **Żadnego morale** — nie istnieje w kodzie (sprawdzone: zero wystąpień). Wszystkie pomysły oparte o nastroje są poza zakresem.
- **Żadnych wyjątków w walce wręcz** („nie zabije bestii”) — to najdroższa opcja z rozważanych, wymaga wyjątków w rozstrzygnięciu walki. Odłożone.

5. RYZYKA

1. **Dzień modłów bez zapłaty** — patrz §3.3. To jest jedyny sposób, w jaki ten projekt może się odwrócić przeciwko własnemu celowi.
2. **Zielona racja za tania** — jeśli będzie tania, wszyscy pakują tylko ją i wiara przestaje cokolwiek znaczyć. Jeśli za droga, wierzący są balastem. Do wyważenia po pierwszym teście na żywo.
3. **Losowanie** — przy sześciu wiarach na małej załodze łatwo o statek, gdzie trzech na czterech ma modły. Rozważyć niższą wagę dla wyznań albo gwarancję, że pierwszy rekrut jest bez wyznania.

6. TESTY (nowe sekcje)

- `faith` jest losowane raz, przeżywa zapis; **stary zapis bez pola dostaje je przy wczytaniu i nie zmienia go przy kolejnym**.
- Wegetarianin odmawia trzech mięsnych racji, szczura i jaja; **zje Green Ration** (warunki takie, że BY zjadł, gdyby nie wiara).
- VENT nad współwyznawcą jest odmawiany z podanym powodem; wymuszenie daje –3 karmy **raz**.
- Dzień modłów: licznik na człowieku, nie globalny — dwóch załogantów zwerbowanych w różnych momentach **nie odpoczywa w tym samym kontrakcie**.

- Powrót z modłów: pełny głód i pełne HP, +2 karmy; zabranie mimo święta: -2 i brak leczenia.
- Czwarta opcja w evencie **pojawia się tylko przy właściwej wierze** i znika przy jej braku (warunki takie, że event BY ją pokazał, gdyby nie warunek).
- „Bez wyznania” nie ma żadnego z powyższych ograniczeń.

MOON WARS — PROJEKT: LISTA GOŃCZA, PIRACI, MISJE (update58)

Dokument projektowy, **zero kodu**. Stan wejściowy sprawdzony w kodzie 2026-09-03 (commit 828ed14 , update53). Zależy od update56 (worek na zwłoki).

1. LISTA GOŃCZA

1.1. Zasady ustalone z graczem

- Lista **przeżywa loty** — pirat pamiętany między kontraktami.
- Start: **jeden pirat**. Z czasem dochodzą kolejni.
- Pirat znika z listy dopiero, gdy zostanie **złapany i dostarczony do bazy**.

1.2. Jeden rejestr

Lista gończa jest **jedynym miejscem, w którym żyje pirat**. Mapa go tylko *znajduje* — nie trzyma drugiej kopii jego danych, nie ma własnego „bossa sektora” o tym samym imieniu. To nasza najstarsza zasada: dwie kopie tej samej liczby zawsze się rozjadą.

W praktyce: rekord piraty (imię, korporacja, poziom, nagroda, stan) siedzi w zapisie meta-progresji, obok bazy i cmentarza. Spotkanie na mapie **wskazuje na rekord**, nie kopiuje go.

Migracja: stary zapis bez listy → lista z jednym wygenerowanym piratem.

1.3. Stany piraty

WANTED	→ na liście, jeszcze nie spotkany
SIGHTED	→ widziany w sektorze, uciekł albo nie doszło do walki
KILLED	→ martwy, ciało na Twoim statku (worek, 2 pola)
DELIVERED	→ oddany w bazie, znika z listy, nagroda wypłacona

Zabicie nie zamyka sprawy. Trup na polu bitwy to nie dowód. Dopiero worek (update56) i przywiezienie ciała do bazy kończy zlecenie — dwa pola ładowni przez pół sektora, zamiast liczenia trupów w tle. Nagroda za ryzyko, nie za kliknięcie.

1.4. Żywcem

Dostarczenie ściganego **żywego** — wyższa nagroda i **+5 karmy** (projekt-karma-dowodcy.md §3.1). Wymaga jeńca na pokładzie, czyli mechaniki, której dziś nie ma. **Do decyzji: robimy w tej paczce czy później?** Jeśli później — w update58 jest tylko ciało, a opis nagrody już zapowiada wariant żywy.

1.5. Gdzie gracz to widzi

Zakładka albo panel w bazie: portret/ikona, korporacja, poziom, nagroda, stan, ostatnio widziany region. Region pochodzi z warstwy z update55.

2. MISJE I CELE DODATKOWE

2.1. Czym są

Krótkie cele nakładane na normalny kontrakt: *zabij 10 wrogich załogantów, pokonaj bossa, znajdź i zniszcz piratę z listy*. Nagroda w CC i/lub karmie.

2.2. Jak to spiąć bez nowego silnika

Cel to **licznik + warunek + nagroda**, czytający zdarzenia, które gra już generuje (zabicia, wygrane walki, dokowania, dostawy). Jeden rejestr aktywnych celów w przelocie, wyczyszczony przy powrocie do bazy.

Zasada: **cel nigdy nie zmienia rozgrywki, tylko ją punktuje**. Misja, która wymusza inny sposób grania, to kontrakt, a nie cel dodatkowy.

2.3. Wyświetlanie

Jedna linia na HUD, składana. Nie robimy z tego drugiego ekranu.

3. RYZYKA

- **Dwie kopie piraty.** Największe ryzyko tej paczki. Test musi sprawdzać, że po spotkaniu na mapie zmiana stanu jest widoczna w **jednym** miejscu.
 - **Worek zajmuje ładownię.** Pirat z listy + własny zmarły = 4 pola. Sprawdzić, czy to nie zabija przelotu na małym statku (scout).
 - **Lista rośnie w nieskończoność.** Potrzebny sufit — ilu ściganych naraz.
 - **Nagroda a ekonomia.** Zbyt hojna zamienia grę w polowanie i wysysa kontrakty.
-

4. TESTY (nowe sekcje)

- Lista przeżywa zapis; **stary zapis bez listy dostaje jednego piraty**.
- Jeden rejestr: po zmianie stanu na mapie **nie istnieje drugi rekord** o tym samym id.
- **KILLED** bez worka **nie** kończy zlecenia; dostarczenie ciała do bazy przechodzi w **DELIVERED** i wypłaca raz.
- Ponowne wejście do bazy z tym samym workiem **nie płaci drugi raz**.
- Pirat **DELIVERED** **nie pojawia się już na mapie**.
- Nowi piraci dochodzą z czasem, ale nie ponad sufit.

- Cele dodatkowe: licznik rośnie na właściwym zdarzeniu, nagroda wypłacana raz, cel niewykonany nie płaci nic; **pętle w testach ograniczone** (reguła z kontraktu paczki — zawieszenie jest jedyną awarią, której przebieg łamania nie raportuje).

MOON WARS — BRIEF GRAFICZNY UI (do przekazania modelowi generującemu obrazy)

Dokument roboczy. Sekcje 1-4 to ustalenia dla nas. **Sekcja 6 to gotowe prompty po angielsku do wklejenia.** Inwentarz ekranów sprawdzony w kodzie 2026-09-03 (commit 828ed14).

1. CEL

Gra ma być **czytelniejsza i ładniejsza**, bez zmiany rozgrywki. Kierunek: **księżycowy — czern, biel, szarość**. Kolor przestaje być dekoracją i zaczyna znaczyć.

Kanwa: **1280 × 720, Canvas 2D**. Cały tekst w grze po angielsku.

2. PALETA

2.1. Dlaczego nie sama skala szarości

Gra **komunikuje stan kolorem**: karma zielona/czerwona, głód (#ff2d44 STARVING / #ffb020 hungry / #1aff8c fed), rozkład ciała, ogień, korporacje, zużyte rozkazy. W czystej szarości to wszystko znika i gra robi się ładniejsza, a mniej czytelna — czyli odwrotnie do celu.

Rozwiązanie: **szara baza + cztery kolory zarezerwowane wyłącznie dla stanu.**

2.2. Baza (regolit)

rola	hex
tło ekranu	#0B0B0C
panel	#17181A
panel wyżej / hover	#232427
linia, ramka	#3A3C41
tekst drugorzędny	#8A8D93
tekst zwykły	#D7D9DD
tekst wyróżniony	#F2F3F5

2.3. Cztery kolory stanu (i ani jednego więcej)

kolor	hex	znaczy
bursztyn	#E0A230	uwaga, ostrzeżenie, „zaraz zabraknie”
czerwień	#D9463C	strata, zagrożenie, śmierć
zieleń	#3FA86B	dobrze, gotowe, najedzony
blady błękit	#8FB6D1	interakcja — zaznaczone, klikalne, aktywne

Zasada twarda: jeśli element nie mówi o stanie, jest szary. Kolor bez znaczenia to szum, który zjada czytelność kolorów, które znaczenie mają.

2.4. Problem korporacji

Sześć korporacji ma dziś własne kolory (`CORP_DEFS` w `crew.js`) — w szarości zlewają się w jedno. **Nie rozwiązujemy tego kolorem**, tylko **znakiem**: każda korporacja dostaje własny prosty glif/wzór (pasy, kropki, kąt, pierścień) na ikonie załoganta. Dodatkowo jeden odcień szarości. Rozpoznawalne też dla daltonistów.

3. ZASADY, KTÓRE MAJĄ WEJŚĆ PRZY OKAZJI

1. **Jedna rodzina prostokątów.** Panele mają wspólny promień rogów, wspólną grubość ramki i trzy wysokości: nagłówek, wiersz, przycisk.
2. **Jedna siatka.** Odstępy wielokrotności 8 px.
3. **Trzy stopnie tekstu.** Nagłówek / treść / podpis. Nic pomiędzy.
4. **Ikony zamiast napisów tam, gdzie napis się nie mieści — ale zawsze z podpowiedzią pod myszą** (jak przy ośmiu rozkazach z update53: osiem glifów w rzędzie bez opisu to osiem zagadek).
5. **Stan zużyty = szary i przekreślony**, nie sam szary (reguła z update53: sam szary czyta się jako „jeszcze nie”).
6. **Napisy nie mogą na siebie wchodzić** — mamy test rysowania, który liczy prostokąty napisów i wywala się na nakładkach (update52a). Projekt ma się w nim zmieścić.

4. INWENTARZ EKRANÓW (co trzeba naszkicować)

Sprawdzone w kodzie — to jest pełna lista, nie zgadywanka.

Menu i ramy 1. `MAIN MENU` — `renderer.js:1327` 2. `OUTCOME` — wynik walki/przelotu, `renderer.js:1840`

Baza (`basescreen.js` , zakładki w linii 24) 3. Pasek zakładek: `HANGAR` · `ARMOURY` · `CREW` · `MESS` · `SUPPLY` · `UPGRADES` · `MEMORIAL` + planowana `GATE` 4. `HANGAR` — statki, kupno/sprzedaż, naprawa 5. `ARMOURY` — broń, montaż, sprzedaż 6. `CREW` — lista załogi, rekrutacja, przewijanie 7. `MESS` —

mesa, karty załogantów, kolejka do awansu 8. `SUPPLY` — magazyn, zakupy (He2, rakiety, sondy, racje) 9. `UPGRADES` — drabinki rozbudowy budynków 10. `MEMORIAL` — cmentarz z historią służby 11. `GATE` (*nowa, update55*) — Moon Gate, lista części `0/N`, zablokowana

Okna nakładane 12. `PACK / WAREHOUSE` — siatka ładowni i magazynu, przeciąganie 13. `PROMOTE` — awans, przewijana kolejka 14. `LEVEL UP` — ekran poziomu, wybór bonusu +0,5% 15. `DOSSIER` —teczka dowódcy, `renderer.js:194` 16. `CPU BOARD` — plansza chipów 5×5, kolumna blokady, karma

Lot 17. `MAP` — mapa sektora, mgła, węzły, **region (nowe, update55)** 18. `EVENT POPUP` — `renderer.js:1495`, wybory 19. `STATION` — **to jest DOM w `ui.js`, nie `canvas`** — inna technika, ten sam język wizualny 20. `LOOT SCREEN` — `lootscreen.js`, sortowanie łupu

Walka 21. `COMBAT` — przekrój obu statków, `renderer.js:1464` 22. `HUD` — CC, He2, paski postępu, `renderer.js:531` 23. `CREW PANEL` — ikony załogi, `ui.js:209` 24. `ORDERS PANEL` — jeden panel rozkazów pod załogą, geometria z `Renderer.orderRects()` (*update53*) 25. `BODY MENU` (*nowe, update56*) — `TREAT / VENT / BAG` przy leżącym

5. CZEGO NIE ZMIENIAMY

Układ i pozycje elementów zostają — to jest zmiana **skóry, nie rozkładu**. Każdy przesunięty przycisk to ryzyko rozjazdu z testem trafienia, a geometrię mamy od *update53* w jednym miejscu i chcemy to utrzymać.

6. GOTOWE PROMPTY (do wklejenia)

6.1. Prompt ustawiający styl — wkleić RAZ, na początku rozmowy

You are art-directing the UI for MOON WARS, a 2D sci-fi roguelike in the spirit of FTL. Canvas is exactly 1280x720, top-down cross-section ship combat, pure Canvas2D – flat vector shapes, no photographic textures, no 3D renders, no lens flares.

ART DIRECTION – "lunar":

- Base palette is greyscale, evoking regolith and hard vacuum: background #0B0B0C, panels #17181A, raised panels #232427, borders #3A3C41, muted text #8A8D93, body text #D7D9DD, bright text #F2F3F5.
- EXACTLY four accent colours exist, and each one carries meaning. Never use them decoratively:
 - amber #E0A230 = caution / running low
 - red #D9463C = loss / threat / death
 - green #3FA86B = good / ready / fed
 - pale blue #8FB6D1 = interactive – selected, clickable, active
- Anything that does not communicate a STATE is grey.

LAYOUT RULES:

- 8px spacing grid. One family of rectangles: shared corner radius, shared 1px border, three heights (header, row, button).
- Three text sizes only: header, body, caption.
- Used-up / spent controls are grey AND struck through, never grey alone.
- Labels must never overlap. Everything readable at 100% zoom.
- All in-game text is ENGLISH.

Output: a clean flat UI mockup at 1280x720, as if screenshotted from the running game. No annotations, no rulers, no design-tool chrome.

6.2. Prompty ekranów — po jednym na wiadomość

Każdy poprzedzić zdaniem: Same art direction as before. Now draw:

- **Base — HANGAR** A space-station hangar management screen. Top row of tabs: HANGAR, ARMOURY, CREW, MESS, SUPPLY, UPGRADES, MEMORIAL, GATE – HANGAR active in pale blue. Left: scrollable list of owned ships with small top-down ship thumbnails, name, hull integrity bar. Right: detail panel with stats, BUY / SELL / REPAIR buttons and prices in CC.
- **Base — MEMORIAL** A memorial wall listing dead crew: name, corporation glyph, cause of death, length of service. Sombre, mostly greyscale, a single amber candle-like marker. Scrollable list, one row per name.
- **Base — GATE (locked)** A construction screen for the "MOON GATE": a large schematic of an unfinished ring gate on the left, a parts checklist on the right with counters reading 0/4, 0/2, 0/6 etc. Everything desaturated and visibly locked. One line of caption text at the bottom stating the gate cannot be built in this version. NO build button.
- **CPU BOARD** A 5x5 grid of square sockets – the commander's chip board. Cells to the left of a vertical "wall" column are the good side, cells to the right are the ruthless side, the wall column itself is dead and clearly struck out. Some cells hold small chip icons, some are locked. A karma meter 0-100 above the grid, green at the high end, red at the low end.

- **COMBAT** Two ships in cross-section, player's on the left, enemy's on the right, rooms as rectangles with crew as small circular icons, doors as line segments. Bottom-left: a crew panel of portraits with HP and hunger bars. Directly beneath it a single ORDERS panel: rows of buttons reading SAVE POS, RETURN, OPEN ALL, CLOSE ALL, BOARD, RECALL, a wide RETREAT bar, then a row labelled SPECIAL with up to eight small glyph buttons – some highlighted, some greyed and struck through. Top of screen: CC and fuel readouts and a jump-charge progress bar.
- **BODY MENU** (*nowe*) Close-up of a ship room in cross-section with a fallen crew member on the floor and a small three-button context menu beside the cursor reading TREAT, VENT, BAG. TREAT greyed out. Compact, no more than 160px wide.
- **MAP** A sector map: nodes connected by thin lines, unexplored area under a soft grey fog, the current node marked in pale blue, danger nodes marked in red, the region name "LUNA" set as a header in the corner.
- **EVENT POPUP** A centred dialogue panel over a dimmed map: a title, four lines of body text, and four stacked choice buttons, the fourth one visually distinct as a faith-specific option. Choices show their cost in small caption text.
- **STATION SHOP** An orbital station trading screen, rendered as an HTML-style panel rather than a game canvas: tabbed categories, a stock list with name, quantity, unit price and BUY buttons, player credits top right.
- **LOOT SCREEN** A post-battle salvage screen: a grid-based cargo hold on the left with items occupying 1x1 and 2x1 cells, a loot pile on the right, drag targets highlighted in pale blue, a SELL THE REST button bottom right, remaining space counter above the grid.

6.3. Po otrzymaniu szkiców

Wybieramy jeden kierunek, potem prosimy o **arkusz elementów**: przyciski (normal/hover/aktywny/wyłączony), paski (HP, głód, ładowanie), ikony ośmiu umiejętności, glify sześciu korporacji, ikony rozkazów.

7. PYTANIA DO GRACZA

1. Cztery kolory stanu — pasują, czy wolisz mniej (np. bez zieleni)?
2. Korporacje na glify zamiast kolorów — zgoda?
3. Czy zmieniamy tylko skórę, czy przy okazji wolno przesuwać elementy? (Zalecam: tylko skóra. Przesuwanie to osobna paczka.)